

Hospital Computerized Maintenance Management System (CMMS)



Supervised by/ Eng. Ezz Eldeen

Made By/ Sondos mohammed

1. Abstract

Hospitals rely on medical equipment to provide safe and effective healthcare services. This project presents a web-based Hospital Computerized Maintenance Management System (CMMS) that digitalizes maintenance requests, work orders, device management, and maintenance tracking using the MERN stack.

2. Introduction

Medical equipment maintenance is critical for patient safety. Traditional manual workflows cause delays and poor tracking. The proposed CMMS centralizes maintenance operations for doctors, nurses, biomedical engineers, technicians, and administrators.

3. Problem Definition

Current maintenance processes are often paper-based and inefficient. The system provides secure authentication, device management, maintenance requests, work orders, notifications, dashboards, and reporting.

4. System Architecture & Technologies

React Frontend

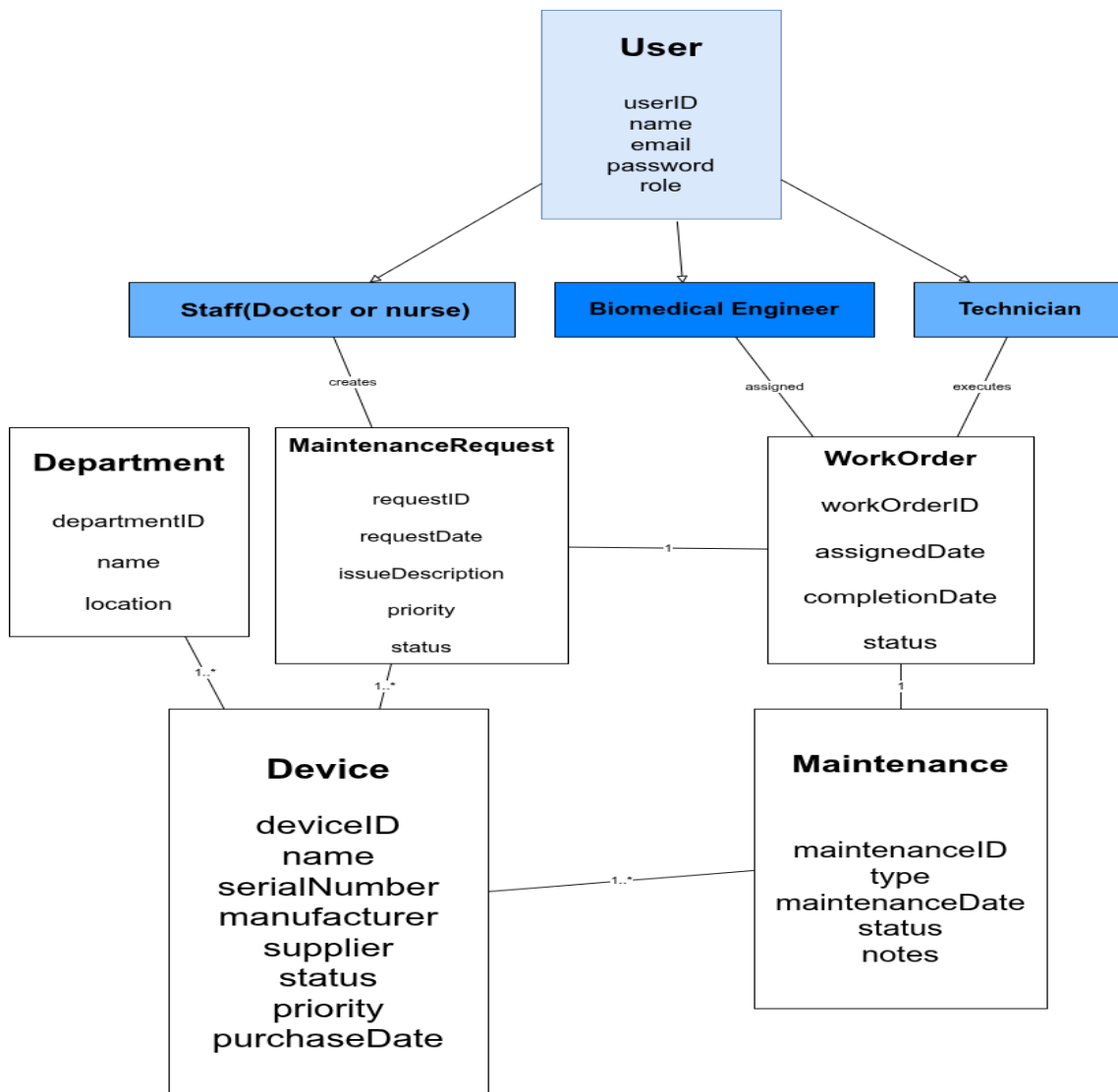
|

REST API (Express)

|
Node.js Backend

|
MongoDB

UML Class Diagram



4.3 User Roles

The system supports four different user roles.

Staff (Doctor/Nurse)

- Login
- Submit maintenance requests
- Track request status
- Receive notifications

Biomedical Engineer

- Review maintenance requests
- Create work orders
- Assign technicians
- Manage devices

Technician

- View assigned work orders
- Perform maintenance
- Update maintenance status

Admin

- Manage users

- Manage roles
 - View reports
 - Configure system
-

4.4 Database Design

The database consists of the following collections:

Users

- userID
 - name
 - email
 - password
 - role
-

Devices

- deviceID
- name
- serialNumber
- manufacturer
- supplier
- status

- priority
 - purchaseDate
-

Departments

- departmentID
 - name
 - location
-

Maintenance Requests

- requestID
 - requestDate
 - issueDescription
 - priority
 - status
-

Work Orders

- workOrderID
- assignedDate
- completionDate

- status

4.6 Technologies Used

Layer	Technology
Frontend	React.js
Backend	Node.js
Framework	Express.js
Database	MongoDB
Authentication	JWT
Password Encryption	bcrypt
API Testing	Postman
Version Control	Git & GitHub
UI Styling	Tailwind CSS

5. Implementation & Dependencies

Backend

Implemented using:

- Node.js
- Express.js

- MongoDB
- Mongoose
- JWT Authentication
- bcrypt
- dotenv
- cors
- multer

Main APIs

- Authentication API
- Users API
- Devices API
- Maintenance Requests API
- Work Orders API
- Notifications API

Frontend

Developed using React.

Implemented pages include:

- Login
- Register
- Dashboard
- Devices
- Work Orders
- Reports
- Notifications
- User Management

- Settings

6. Achievements & Progress

- Authentication,
- Role-based access,
- Device Management,
- Maintenance Requests,
- Dashboard UI,
- Work Orders,
- REST APIs,
- Database Integration,
- Responsive Design,
- Real-time notifications,
- QR codes,
- preventive maintenance scheduling,
- AI-based maintenance prediction,
- PDF/Excel reports,
- cloud deployment.
- Chatbot
- live chat

7. Future Work

➤ Mobile App

8. References

1. MongoDB Documentation: <https://www.mongodb.com/docs/>
2. React Documentation: <https://react.dev/>
3. Express.js Documentation: <https://expressjs.com/>
4. Node.js Documentation: <https://nodejs.org/>
5. Mongoose Documentation: <https://mongoosejs.com/>
6. JWT Documentation: <https://jwt.io/>
7. Tailwind CSS Documentation: <https://tailwindcss.com/>
8. GitHub Repo: https://github.com/sondosmohammed0100-cmyk/Graduation_Proj